```
/* 1
#include <iostream>
using namespace std;

// ---------- Searching Algorithms ----------

// Linear Search
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

// Sentinel Search
int sentinelSearch(int arr[], int n, int key) {
    int last = arr[n - 1];
    arr[n - 1] = key; // place sentinel

    int i = 0;
    while (arr[i] != key)
        i++;

    arr[n - 1] = last; // restore last element
    if (i < n - 1 || arr[n - 1] == key)
        return i;
    return -1;
}
```

```
// Binary Search (works only on sorted arrays)
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

// Fibonacci Search (works on sorted arrays)
int min(int a, int b) { return (a < b) ? a : b; }

int fibonacciSearch(int arr[], int n, int key) {
    int fib2 = 0, fib1 = 1;
    int fib = fib1 + fib2;

    while (fib < n) {
        fib2 = fib1;
        fib1 = fib;
        fib = fib1 + fib2;
    }

    int offset = -1;
    while (fib > 1) {
```

```
        int i = min(offset + fib2, n - 1);
        if (arr[i] < key) {
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        }
        else if (arr[i] > key) {
            fib = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
        }
        else
            return i;
    }
    if (fib1 && arr[offset + 1] == key)
        return offset + 1;
    return -1;
}

// Interpolation Search (works best for uniformly distributed sorted arrays)
int interpolationSearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high && key >= arr[low] && key <= arr[high]) {
        if (low == high) {
            if (arr[low] == key) return low;
            return -1;
        }
```

```
        int pos = low + ((double)(high - low) /
                  (arr[high] - arr[low])) * (key - arr[low]);

        if (arr[pos] == key)
            return pos;
        if (arr[pos] < key)
            low = pos + 1;
        else
            high = pos - 1;
    }
    return -1;
}

// ---------- Main Function ----------
int main() {
    int n, key, choice;

    cout << "Enter number of items in bill: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " item prices:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter item price to search: ";
    cin >> key;

    cout << "\nChoose Searching Algorithm:\n";
```

```
cout << "1. Linear Search\n";
cout << "2. Sentinel Search\n";
cout << "3. Binary Search (sorted array required)\n";
cout << "4. Fibonacci Search (sorted array required)\n";
cout << "5. Interpolation Search (sorted array required)\n";
cout << "Enter choice: ";
cin >> choice;

int index = -1;

switch (choice) {
    case 1:
        index = linearSearch(arr, n, key);
        break;
    case 2:
        index = sentinelSearch(arr, n, key);
        break;
    case 3:
        index = binarySearch(arr, n, key);
        break;
    case 4:
        index = fibonacciSearch(arr, n, key);
        break;
    case 5:
        index = interpolationSearch(arr, n, key);
        break;
    default:
        cout << "Invalid choice!";
        return 0;
}
```

```
    if (index != -1)
        cout << "\nItem found at index " << index << " (0-based index)\n";
    else
        cout << "\nItem not found in bill!\n";

    return 0;
}

*/



/* 2
#include <iostream>
#include <string>
using namespace std;

class Product {
public:
    int id;
    string name;
    string manufacturer;
    float price;
    int rating; // out of 5
};

void displayProducts(Product arr[], int n) {
    if (n == 0) {
        cout << "\nNo products available.\n";
        return;
```

```
    }
    cout << "\nProduct ID\tName\tManufacturer\tPrice\tRating\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i].id << "\t\t"
             << arr[i].name << "\t"
             << arr[i].manufacturer << "\t"
             << arr[i].price << "\t"
             << arr[i].rating << endl;
    }
}

// Bubble Sort (by ID ascending)
void bubbleSortByID(Product arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j].id > arr[j + 1].id) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
    cout << "\nProducts sorted by Product ID (Bubble Sort).\n";
}

// Selection Sort (by Price ascending)
void selectionSortByPrice(Product arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j].price < arr[minIndex].price) {
                minIndex = j;
```

```
            }
        }
        swap(arr[i], arr[minIndex]);
    }
    cout << "\nProducts sorted by Price (Selection Sort).\n";
}

// Insertion Sort (by Rating descending)
void insertionSortByRating(Product arr[], int n) {
    for (int i = 1; i < n; i++) {
        Product key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].rating < key.rating) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "\nProducts sorted by Rating (Insertion Sort).\n";
}

int main() {
    Product products[100]; // max 100 products
    int n = 0; // number of products
    int choice;

    do {
        cout << "\n--- E-Commerce Product Menu ---\n";
        cout << "1. Add Product Details\n";
        cout << "2. Display Product Details\n";
```

```
cout << "3. Sort by Product ID (Bubble Sort)\n";
cout << "4. Sort by Product Price (Selection Sort)\n";
cout << "5. Sort by Product Rating (Insertion Sort)\n";
cout << "6. Exit\n";
cout << "Enter your choice: ";
cin >> choice;

switch (choice) {
case 1: {
    cout << "\nEnter number of products to add: ";
    int count; cin >> count;
    for (int i = 0; i < count; i++) {
        cout << "\nEnter details of product " << (n + 1) << ":\n";
        cout << "Product ID: "; cin >> products[n].id;
        cout << "Name: "; cin >> products[n].name;
        cout << "Manufacturer: "; cin >> products[n].manufacturer;
        cout << "Price: "; cin >> products[n].price;
        cout << "Quality Rating (1-5): "; cin >> products[n].rating;
        n++;
    }
    break;
}
case 2:
    displayProducts(products, n);
    break;
case 3:
    bubbleSortByID(products, n);
    displayProducts(products, n);
    break;
case 4:
```

```
            selectionSortByPrice(products, n);
            displayProducts(products, n);
            break;
        case 5:
            insertionSortByRating(products, n);
            displayProducts(products, n);
            break;
        case 6:
            cout << "Exiting program...\n";
            break;
        default:
            cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 6);

    return 0;
}

*/



/* 3
#include <iostream>
#include <string>
using namespace std;

const int TABLE_SIZE = 10;

class Client {
```

```
public:
    string name;
    string phone;

    Client() {
        name = "";
        phone = "";
    }
};

class HashTable {
private:
    Client table[TABLE_SIZE];

    int hashFunction(const string &key) {
        int sum = 0;
        for(char c : key)
            sum += c;
        return sum % TABLE_SIZE;
    }

public:
    void insert(const string &name, const string &phone) {
        int index = hashFunction(name);

        for(int i = 0; i < TABLE_SIZE; i++) {
            int newIndex = (index + i) % TABLE_SIZE;
            if(table[newIndex].name == "") {
                table[newIndex].name = name;
                table[newIndex].phone = phone;
```

```
            cout << "Inserted at index " << newIndex << endl;
            return;
        }
    }
    cout << "Hash table full!\n";
}

void search(const string &name) {
    int index = hashFunction(name);

    for(int i = 0; i < TABLE_SIZE; i++) {
        int newIndex = (index + i) % TABLE_SIZE;
        if(table[newIndex].name == name) {
            cout << "Found at index " << newIndex
                 << " → Phone: " << table[newIndex].phone << endl;
            return;
        }
        if(table[newIndex].name == "") break; // stop searching
    }

    cout << "Client not found.\n";
}

void display() {
    cout << "\n--- Telephone Book ---\n";
    for(int i = 0; i < TABLE_SIZE; i++) {
        cout << "Index " << i << ": ";
        if(table[i].name == "")
            cout << "Empty\n";
        else
```

```
            cout << table[i].name << " - " << table[i].phone << endl;
        }
    }
};

int main() {
    HashTable ht;
    int choice;
    string name, phone;

    do {
        cout << "\nMenu:\n1. Insert\n2. Display\n3. Search\n4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();

        if(choice == 1) {
            cout << "Enter name: ";
            getline(cin, name);
            cout << "Enter phone: ";
            getline(cin, phone);
            ht.insert(name, phone);
        }
        else if(choice == 2) {
            ht.display();
        }
        else if(choice == 3) {
            cout << "Enter name to search: ";
            getline(cin, name);
            ht.search(name);
```

```
        }

    } while(choice != 4);

    return 0;
}

*/



/* 4

#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

class node {
public:
    int music_id;
    string title;
    float time_duration;
    node* next;

    node(int id, string t, float dur) {
        music_id = id;
        title = t;
        time_duration = dur;
        next = nullptr;
    }
};

class website {
private:
    node* head;
    node* tail;

public:
    website() {
        head = nullptr;
```

```
    tail = nullptr;
}

// INSERT AT ANY POSITION (circular)
void insertAtPosition(int pos, int id, string title, float dur) {
    node* newNode = new node(id, title, dur);

    if (head == nullptr) {
        head = tail = newNode;
        newNode->next = head;
        return;
    }

    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        tail->next = head;
        return;
    }

    node* temp = head;
    for (int i = 1; i < pos - 1 && temp->next != head; i++)
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;

    if (temp == tail)
        tail = newNode;
}

// DELETE AT ANY POSITION
void deleteAtPosition(int pos) {
    if (head == nullptr) return;

    node* temp = head;

    if (pos == 1) {
        if (head == tail) {
            delete head;
            head = tail = nullptr;
            return;
```

```
        }
        head = head->next;
        tail->next = head;
        delete temp;
        return;
    }

    node* prev = nullptr;
    for (int i = 1; i < pos && temp->next != head; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == tail)
        tail = prev;

    prev->next = temp->next;
    delete temp;
}

// SORT
void sortBy(string field) {
    if (head == nullptr || head->next == head) return;

    for (node* i = head; i->next != head; i = i->next) {
        for (node* j = i->next; j != head; j = j->next) {

            bool swapNeeded = false;

            if (field == "music_id" && i->music_id > j->music_id)
                swapNeeded = true;
            else if (field == "title" && i->title > j->title)
                swapNeeded = true;
            else if (field == "duration" && i->time_duration > j->time_duration)
                swapNeeded = true;

            if (swapNeeded) {
                swap(i->music_id, j->music_id);
                swap(i->title, j->title);
                swap(i->time_duration, j->time_duration);
            }
        }
    }
```

```
    }

    // LOOP PLAYING
    void playLoop(int cycles) {
        if (head == nullptr) return;

        node* temp = head;

        while (cycles--) {
            do {
                cout << "Playing: " << temp->music_id << " | "
                     << temp->title << " | "
                     << temp->time_duration << " mins\n";
                temp = temp->next;
            } while (temp != head);
        }
    }

    // DISPLAY PLAYLIST
    void display() {
        if (head == nullptr) {
            cout << "Playlist is empty.\n";
            return;
        }

        node* temp = head;
        do {
            cout << temp->music_id << " | "
                 << temp->title << " | "
                 << temp->time_duration << endl;
            temp = temp->next;
        } while (temp != head);
    }
};

int main() {
    website w;
    int choice;

    while (true) {
        cout << "\n===== PLAYLIST MENU =====\n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
```

```
cout << "3. Display\n";
cout << "4. Sort\n";
cout << "5. Play Loop\n";
cout << "6. Exit\n";
cout << "Enter your choice: ";
cin >> choice;


if (choice == 1) {
    int pos, id;
    string title;
    float duration;


    cout << "Enter position: ";
    cin >> pos;
    cout << "Enter music ID: ";
    cin >> id;
    cout << "Enter title: ";
    cin.ignore();
    getline(cin, title);
    cout << "Enter duration: ";
    cin >> duration;


    w.insertAtPosition(pos, id, title, duration);
    cout << "Inserted successfully.\n";
}


else if (choice == 2) {
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;


    w.deleteAtPosition(pos);
    cout << "Deleted successfully.\n";
}


else if (choice == 3) {
    cout << "\nPlaylist:\n";
    w.display();
}


else if (choice == 4) {
    cout << "Sort by (music_id/title/duration): ";
    string field;
```

```
            cin >> field;

            w.sortBy(field);
            cout << "Sorted successfully.\n";
        }

        else if (choice == 5) {
            int cycles;
            cout << "Enter number of cycles: ";
            cin >> cycles;

            w.playLoop(cycles);
        }

        else if (choice == 6) {
            cout << "Exiting...\n";
            break;
        }

        else {
            cout << "Invalid choice. Try again.\n";
        }
    }

    return 0;
}




*/

/* 5

#include <iostream>
#include <cstring>
using namespace std;

class Node {
public:
    int id;
    char desc[20];
    char time[20];
```

```
    Node* next;
    Node* prev;


    Node(int pid, char* d, char* t) {
        id = pid;
        strcpy(desc, d);
        strcpy(time, t);
        next = prev = NULL;
    }
};


class Browser {
private:
    Node* head;
    Node* tail;
    Node* cur;


public:
    Browser() {
        head = tail = cur = NULL;
    }


    // Add page to end
    void add(int id, char* d, char* t) {
        Node* n = new Node(id, d, t);


        if (head == NULL) {
            head = tail = cur = n;
        } else {
            tail->next = n;
            n->prev = tail;
            tail = n;
        }
        cout << "Page added!\n";
    }


    // Visit page by ID
    void visit(int id) {
        Node* temp = head;
        while (temp != NULL) {
            if (temp->id == id) {
                cur = temp;
                cout << "Visiting: " << temp->desc << " (" << temp->time << ")\n";
                return;
```

```
        }
        temp = temp->next;
    }
    cout << "Page not found!\n";
}

// Backward
void back() {
    if (cur == NULL || cur->prev == NULL) {
        cout << "No previous page!\n";
        return;
    }
    cur = cur->prev;
    cout << "Back to: " << cur->desc << "\n";
}

// Forward
void forward() {
    if (cur == NULL || cur->next == NULL) {
        cout << "No next page!\n";
        return;
    }
    cur = cur->next;
    cout << "Forward to: " << cur->desc << "\n";
}

// Delete page by ID
void del(int id) {
    Node* temp = head;

    while (temp != NULL) {
        if (temp->id == id) {

            if (temp->prev) temp->prev->next = temp->next;
            if (temp->next) temp->next->prev = temp->prev;

            if (temp == head) head = temp->next;
            if (temp == tail) tail = temp->prev;
            if (temp == cur) cur = NULL;

            delete temp;
            cout << "Page deleted!\n";
            return;
```

```
            }
            temp = temp->next;
        }
        cout << "Page not found!\n";
    }

    // Print from start
    void showForward() {
        Node* temp = head;
        if (temp == NULL) {
            cout << "No history!\n";
            return;
        }

        cout << "\nHistory (Forward):\n";
        while (temp != NULL) {
            cout << temp->id << ": " << temp->desc << " (" << temp->time << ")\n";
            temp = temp->next;
        }
    }

    // Print from end
    void showBackward() {
        Node* temp = tail;
        if (temp == NULL) {
            cout << "No history!\n";
            return;
        }

        cout << "\nHistory (Backward):\n";
        while (temp != NULL) {
            cout << temp->id << ": " << temp->desc << " (" << temp->time << ")\n";
            temp = temp->prev;
        }
    }
};

int main() {
    Browser b;
    int ch, id;
    char d[20], t[20];

    do {
        cout << "\n--- Browser Menu ---\n";
```

```
cout << "1. Add Page\n";
cout << "2. Visit Page\n";
cout << "3. Back\n";
cout << "4. Forward\n";
cout << "5. Delete Page\n";
cout << "6. Show Forward\n";
cout << "7. Show Backward\n";
cout << "8. Exit\n";
cout << "Enter choice: ";
cin >> ch;


switch (ch) {
   case 1:
      cout << "Page ID: ";
      cin >> id;
      cin.ignore();
      cout << "Description: ";
      cin.getline(d, 20);
      cout << "Timestamp: ";
      cin.getline(t, 20);
      b.add(id, d, t);
      break;


   case 2:
      cout << "Enter ID to visit: ";
      cin >> id;
      b.visit(id);
      break;


   case 3:
      b.back();
      break;


   case 4:
      b.forward();
      break;


   case 5:
      cout << "Enter ID to delete: ";
      cin >> id;
      b.del(id);
      break;


   case 6:
```

```
            b.showForward();
            break;


        case 7:
            b.showBackward();
            break;


        case 8:
            cout << "Exiting...\n";
            break;


        default:
            cout << "Invalid choice!\n";
      }


   } while (ch != 8);
}



*/

/* 6

#include<iostream>
using namespace std;


class Seat
{
   public :
      bool booked;
      Seat* prev;
      Seat* next;


      Seat()
      {
         booked = false;
         prev = NULL;
         next = NULL;
      }
};

class Row
{
```

```
    public :
        Seat* head;
        int totalseats;

        Row()
        {
            head = NULL;
            totalseats = 0;
        }
        Row(int n)
        {
            head = NULL;
            totalseats = n;


            Seat *p = NULL;
            for(int i = 0; i < n; i++)
            {
                Seat* q = new Seat();


                if(head == NULL)
                {
                    head = q;
                    p = head;
                }
                else
                {
                    p -> next = q;
                    q -> prev = p;
                    p = q;
                }
                head -> prev = p;
                p -> next = head;
            }
        }
};


class Theater
{
    public :
        Row** row;
        int totalrows;


        Theater()
        {
            row = NULL;
```

```
        totalrows= 0;
    }

    Theater(int rowscount, int totalseats)
    {
        totalrows = rowscount;
        row = new Row*[rowscount];

        for(int i = 0; i < rowscount; i++)
        {
            row[i] = new Row(totalseats);
        }
    }

    void displayAvailableSeats();
    void bookSeat(int, int);
    void cancelBooking(int, int);
};

int main()
{
    int rows, seats;

    cout << "Enter number of rows : ";
    cin >> rows;
    cout << "Enter number of seats per row : ";
    cin >> seats;

    Theater theater (rows, seats);

    int choice;

    cout << "\n1. Display Available Seats\n";
    cout << "2. Book a Seat\n";
    cout << "3. Cancel Booking\n";
    cout << "4. Exit\n";

    do
    {
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
```

```
        {
            case 1:
                theater.displayAvailableSeats();
                break;


            case 2:
            {
                int rowNum, seatNum;
                cout << "Enter row number to book: ";
                cin >> rowNum;
                cout << "Enter seat number to book: ";
                cin >> seatNum;
                theater.bookSeat(rowNum, seatNum);

                break;
            }


            case 3:
            {
                int rowNum, seatNum;
                cout << "Enter row number to cancel: ";
                cin >> rowNum;
                cout << "Enter seat number to cancel: ";
                cin >> seatNum;
                theater.cancelBooking(rowNum, seatNum);

                break;
            }


            case 4:
                cout << "Exiting the program.\n";
                break;


            default:
                cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 4);


    return 0;
}

void Theater::displayAvailableSeats()
{
    for(int i = 0; i < totalrows; i++)
    {
```

```
        cout << "Row " << i+1 << " : ";
        Seat* p = row[i]->head;


        for(int j = 0; j < row[i]->totalseats; j++)
        {
            if(p->booked == false)
            {
                cout << "S" << j + 1 << " ";
            }
            else
            {
                cout << "   ";  // show 'b' for booked seats
            }
            p = p->next;
        }
        cout << endl;
    }
}



void Theater:: bookSeat(int rownum, int seatnum)
{
    if(rownum < 1 || rownum > totalrows)
    {
        cout << "Invalid Row";
        return;
    }


    Row* Rows = row[rownum-1];
    if(seatnum < 1 || seatnum > Rows -> totalseats)
    {
        cout << "Invalid seats";
        return;
    }


    Seat* p = Rows -> head;
    for(int i = 0; i < seatnum-1; i++)
    {
        p = p -> next;
    }
    if(p -> booked == true)
    {
        cout << "Seat already booked!" << endl;
    }
    else
```

```
    {
        p -> booked = true;
        cout << "Seat " << seatnum << " in Row " << rownum << " is successfully booked." << endl;
    }
}


void Theater :: cancelBooking(int rownum, int seatnum)
{
    if(rownum < 1 || rownum > totalrows)
    {
        cout << "Invalid Row";
        return;
    }


    Row* Rows = row[rownum-1];
    if(seatnum < 1 || seatnum > Rows -> totalseats)
    {
        cout << "Invalid seats";
        return;
    }


    Seat* p = Rows -> head;
    for(int i = 0; i < seatnum-1; i++)
    {
        p = p -> next;
    }
    if(p -> booked == false)
    {
        cout << "Seat is not booked yet!" << endl;
    }
    else
    {
        p -> booked = false;
        cout << "Booking for Seat " << seatnum << " in Row " << rownum << " has been cancelled." << endl;
    }
}


*/

/* 7

#include <iostream>
#include <string>
using namespace std;


#define MAX 200
```

```
// ============================
// Stack Class (Array-Based)
// ============================
class Stack {
private:
    char arr[MAX];
    int top;


public:
    Stack() { top = -1; }


    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == MAX - 1; }


    void push(char x) {
        if (isFull()) {
            cout << "Error: Stack Overflow!" << endl;
            return;
        }
        arr[++top] = x;
    }


    char pop() {
        if (isEmpty()) {
            return '\0';
        }
        return arr[top--];
    }


    char peek() const {
        if (isEmpty()) return '\0';
        return arr[top];
    }
};


// ============================
// Utility Functions
// ============================
bool isOpening(char ch) {
    return ch == '(' || ch == '{' || ch == '[';
}
```

```
bool isClosing(char ch) {
    return ch == ')' || ch == '}' || ch == ']';
}


bool isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}


// ============================
// Main Checker Function
// ============================
bool isWellParenthesized(const string &exp) {
    Stack st;


    for (char ch : exp) {


        // Ignore all non-bracket characters (variables, operators, spaces, numbers, etc.)
        if (isOpening(ch)) {
            st.push(ch);
        }
        else if (isClosing(ch)) {
            if (st.isEmpty()) return false;


            char top = st.pop();
            if (!isMatchingPair(top, ch))
                return false;
        }
    }


    // Must be empty at the end
    return st.isEmpty();
}


// ============================
// Driver Code
// ============================
int main() {
    string expression;


    cout << "Enter an expression: ";
    getline(cin, expression);  // supports spaces
```

```
    if (isWellParenthesized(expression))
        cout << "Expression is well parenthesized." << endl;
    else
        cout << "Expression is NOT well parenthesized." << endl;

    return 0;
}



*/

/* 8

#include <iostream>
using namespace std;


class Deque {
private:
    int arr[50];
    int front, rear, size;


public:
    Deque(int n) {
        size = n;
        front = rear = -1;
    }


    bool isFull() {
        return ((front == 0 && rear == size - 1) || (front == rear + 1));
    }


    bool isEmpty() {
        return (front == -1);
    }


    void insertFront(int x) {
        if (isFull()) {
            cout << "Deque is Full!\n";
            return;
        }
```

```
    if (front == -1) {  // first element
        front = rear = 0;
    }
    else if (front == 0) {
        front = size - 1;
    }
    else {
        front--;
    }

    arr[front] = x;
    cout << "Inserted at Front: " << x << endl;
}

void insertRear(int x) {
    if (isFull()) {
        cout << "Deque is Full!\n";
        return;
    }

    if (front == -1) { // first element
        front = rear = 0;
    }
    else if (rear == size - 1) {
        rear = 0;
    }
    else {
        rear++;
    }

    arr[rear] = x;
    cout << "Inserted at Rear: " << x << endl;
}

void deleteFront() {
    if (isEmpty()) {
        cout << "Deque is Empty!\n";
        return;
    }

    cout << "Deleted from Front: " << arr[front] << endl;

    if (front == rear) { // only one element
```

```
            front = rear = -1;
        }
        else if (front == size - 1) {
            front = 0;
        }
        else {
            front++;
        }
    }

    void deleteRear() {
        if (isEmpty()) {
            cout << "Deque is Empty!\n";
            return;
        }

        cout << "Deleted from Rear: " << arr[rear] << endl;

        if (front == rear) { // only one element
            front = rear = -1;
        }
        else if (rear == 0) {
            rear = size - 1;
        }
        else {
            rear--;
        }
    }

    void display() {
        if (isEmpty()) {
            cout << "Deque is Empty!\n";
            return;
        }

        cout << "Deque Elements: ";
        int i = front;
        while (true) {
            cout << arr[i] << " ";
            if (i == rear) break;
            i = (i + 1) % size;
        }
        cout << endl;
    }
};
```

```
// ----------------- MAIN (MENU DRIVEN) -------------------

int main() {
    int n, choice, value;

    cout << "Enter size of Deque: ";
    cin >> n;

    Deque dq(n);

    while (true) {
        cout << "\n===== MENU =====\n";
        cout << "1. Insert at Front\n";
        cout << "2. Insert at Rear\n";
        cout << "3. Delete from Front\n";
        cout << "4. Delete from Rear\n";
        cout << "5. Display\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: ";
                cin >> value;
                dq.insertFront(value);
                break;

            case 2:
                cout << "Enter value: ";
                cin >> value;
                dq.insertRear(value);
                break;

            case 3:
                dq.deleteFront();
                break;

            case 4:
                dq.deleteRear();
                break;
```

```
            case 5:
                dq.display();
                break;

            case 6:
                cout << "Exiting...\n";
                return 0;

            default:
                cout << "Invalid Choice!\n";
        }
    }
}



*/


/* 9

#include <iostream>
#include <queue>
#include <stack>
using namespace std;


struct Node {
    int data;
    Node* left;
    Node* right;


    Node(int value) {
        data = value;
        left = right = NULL;
    }
};


// Insert into BST
Node* insert(Node* root, int value) {
    if (root == NULL)
        return new Node(value);
```

```
    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

// Preorder Traversal
void preorder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

// Inorder Traversal
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Postorder Traversal
void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}

// BFS using queue
void bfs(Node* root) {
    if (!root) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* temp = q.front();
        q.pop();
```

```
        cout << temp->data << " ";

        if (temp->left) q.push(temp->left);
        if (temp->right) q.push(temp->right);
    }
}

// DFS using stack
void dfs(Node* root) {
    if (!root) return;

    stack<Node*> st;
    st.push(root);

    while (!st.empty()) {
        Node* temp = st.top();
        st.pop();

        cout << temp->data << " ";

        // Push right first so left is processed first
        if (temp->right) st.push(temp->right);
        if (temp->left) st.push(temp->left);
    }
}

// Height of tree (longest path from root)
int height(Node* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

// Find minimum value in BST
int findMin(Node* root) {
    if (root == NULL) {
        cout << "Tree is empty.\n";
        return -1;
    }
    while (root->left != NULL)
        root = root->left;

    return root->data;
```

```
}

// Mirror the tree (swap left and right)
void mirror(Node* root) {
    if (root == NULL) return;

    swap(root->left, root->right);

    mirror(root->left);
    mirror(root->right);
}

// Search a value in BST
bool searchValue(Node* root, int value) {
    if (root == NULL) return false;

    if (root->data == value) return true;
    else if (value < root->data) return searchValue(root->left, value);
    else return searchValue(root->right, value);
}

// Main
int main() {
    Node* root = NULL;
    int n, value;

    cout << "Enter number of values to insert: ";
    cin >> n;

    cout << "Enter values: ";
    for (int i = 0; i < n; i++) {
        cin >> value;
        root = insert(root, value);
    }

    cout << "\nPreorder: ";
    preorder(root);

    cout << "\nInorder: ";
    inorder(root);
```

```
    cout << "\nPostorder: ";
    postorder(root);

    cout << "\nBFS (Level Order): ";
    bfs(root);

    cout << "\nDFS (Stack-based): ";
    dfs(root);

    cout << "\n\nHeight (Longest path from root): " << height(root);

    cout << "\nMinimum value: " << findMin(root);

    cout << "\n\nMirroring tree...\n";
    mirror(root);

    cout << "Inorder after mirroring: ";
    inorder(root);

    int key;
    cout << "\n\nEnter value to search: ";
    cin >> key;

    if (searchValue(root, key))
        cout << "Value found!\n";
    else
        cout << "Value not found.\n";

    return 0;
}



*/

/* 10

#include <iostream>
#include <string>
using namespace std;

// Node structure for file/folder
```

```
struct Node {
    string name;
    bool isFile;   // true if it's a file, false if it's a folder
    Node* left;    // first child
    Node* right;   // next sibling


    Node(string n, bool file) : name(n), isFile(file), left(nullptr), right(nullptr) {}
};


// Function to print tree in hierarchical manner
void printTree(Node* root, string indent = "") {
    if (!root) return;
    cout << indent << (root->isFile ? "File: " : "Folder: ") << root->name << endl;
    printTree(root->left, indent + "    ");  // children
    printTree(root->right, indent);          // siblings
}


// Function to build a folder and all its children recursively
Node* buildNode() {
    string name;
    int type;


    cout << "Enter name: ";
    cin >> name;


    cout << "Enter type (0 = Folder, 1 = File): ";
    cin >> type;


    Node* node = new Node(name, type == 1);


    // If it is a file, no children
    if (node->isFile) return node;


    // If folder → ask for number of children
    int n;
    cout << "How many children does folder '" << name << "' have? ";
    cin >> n;


    Node* prev = nullptr;
    for (int i = 0; i < n; i++) {
        cout << "\n--- Child " << i + 1 << " of " << name << " ---\n";
        Node* child = buildNode();
```

```
        if (i == 0)
            node->left = child;  // first child
        else
            prev->right = child; // next sibling

        prev = child;
    }

    return node;
}

int main() {
    cout << "Build your directory tree:\n\n";

    cout << "Create root folder:\n";
    Node* root = buildNode();

    cout << "\n\n*** Directory Tree Structure ***\n";
    printTree(root);

    return 0;
}

*/
```